

TECHNICAL STANDARD OPERATING PROCEDURE

**PostgreSQL 18
Performance Tuning Views & Functions**

Practical DBA Monitoring, Diagnosis and Validation Guide

Technology / Platform	PostgreSQL
Target Version	PostgreSQL 18
Document Type	Technical SOP / Performance Reference
Document Version	1.0
Prepared Date	27 August 2026
Classification	Technical Training / Operations Reference

Document Control

Field	Value
Document Title	PostgreSQL 18 Performance Tuning Views & Functions
Purpose	Provide a production-oriented reference for PostgreSQL statistics views, supplied extensions and performance diagnostics.
Scope	Core PostgreSQL 18 monitoring views plus selected supplied extensions used by DBAs for performance analysis.
Primary Source	PostgreSQL 18 Official Documentation (postgresql.org)
Intended Audience	PostgreSQL DBAs, SREs, platform engineers, database engineers and technical learners.
Review Trigger	PostgreSQL major-version change or material monitoring/statistics behavior change.

Revision History

Version	Date	Description
1.0	27 August 2026	Initial release based on supplied performance-view matrix and PostgreSQL 18 official documentation.

Important: `pg_stat_statements` is a supplied extension that must be configured and created in each database where its SQL interface is required. `pgstattuple()` and `pgstatindex()` are functions from the supplied `pgstattuple` extension. They are not ordinary built-in statistics views.

Table of Contents / Index

1. Introduction	3
2. Corrected Performance View Matrix	3
3. Prerequisites and Baseline Setup	5
4. Core Table and I/O Statistics	6
5. Index Statistics and Index Health	7
6. Sessions, Locks and Blocking	9
7. Query Performance with pg_stat_statements	10
8. Database, Vacuum and Index Build Progress	10
9. Replication and WAL Monitoring	11
10. Additional High-Value PostgreSQL 18 Views	12
11. Practical Performance Tuning Workflow	13
12. Production Troubleshooting Scenarios	14
13. Production Readiness Checklist	15
14. Operational Cheat Sheet	15
15. Official References	16

1. Introduction

PostgreSQL exposes both dynamic activity views and cumulative statistics views that help a DBA understand workload, I/O behavior, table and index usage, background maintenance, replication and WAL activity. Performance tuning should not rely on one ratio or one view. The correct method is to combine database statistics, query plans, operating-system metrics and workload context.

1.1 Purpose

This document turns the supplied list of important PostgreSQL performance views into an operational SOP. It explains what each object measures, what it does not prove, how to query it safely, and how to combine the results during a real incident or tuning exercise.

1.2 Key principles

- Statistics are evidence, not automatic diagnoses. A low cache-hit ratio, unused index or dead-tuple count must be interpreted with workload age and business context.
- Many cumulative counters are meaningful only when you know when statistics were reset or when the server started.
- PostgreSQL I/O statistics count PostgreSQL buffer hits and reads, but cannot tell whether an operating-system read came from physical storage or the kernel page cache.
- Always use EXPLAIN (ANALYZE, BUFFERS) carefully on representative queries when query-plan diagnosis is required; ANALYZE executes the statement.
- Production monitoring should combine PostgreSQL views with OS/cloud metrics for CPU, memory, storage latency, IOPS and network.

2. Corrected Performance View Matrix

#	Area	Primary View / Function	Correct Purpose	Type
1	Table statistics	pg_stat_user_tables	Table scans, tuple changes, estimated live/dead rows, VACUUM/ANALYZE history.	Core view
2	Table I/O statistics	pg_statio_user_tables	PostgreSQL buffer reads/hits for heap, indexes and TOAST; useful for cache-effectiveness analysis.	Core view
3	Table physical health	pgstattuple()	Physical relation length, live/dead tuple space and free space; useful for bloat investigation.	pgstattuple extension
4	Index statistics	pg_stat_user_indexes	Index scans and tuples read/fetched; useful for index-usage analysis.	Core view
5	Index I/O statistics	pg_statio_user_indexes	Index block reads and buffer hits.	Core view
6	B-tree index health	pgstatindex()	B-tree page statistics such as average leaf density and leaf fragmentation; not a generic index-fragmentation function.	pgstattuple extension
7	Index metadata	pg_index + pg_indexes	Validity, uniqueness, readiness, key/include metadata and reconstructed definitions. Duplicate detection requires comparison logic.	Catalog + system view
8	Sessions	pg_stat_activity	Current sessions/backend processes, state, wait events, query text and transaction timing.	Dynamic view
9	Locks	pg_locks + pg_blocking_pids()	Held/waiting locks and blocker relationships.	Core view + function
10	Query performance	pg_stat_statements	Aggregated planning/execution statistics by normalized statement/query ID.	Supplied extension
11	Database statistics	pg_stat_database	Database-level transactions,	Core view

#	Area	Primary View / Function	Correct Purpose	Type
			tuples, block reads/hits, temp activity, deadlocks and session statistics.	
12	VACUUM progress	pg_stat_progress_vacuum	Live progress for running VACUUM operations.	Dynamic progress view
13	Index build progress	pg_stat_progress_create_index	Live progress for CREATE INDEX and REINDEX.	Dynamic progress view
14	Replication	pg_stat_replication	One row per WAL sender; useful for streaming-replication state and LSN positions.	Dynamic view
15	WAL	pg_stat_wal	Instance-wide WAL records, full-page images, WAL bytes and related WAL counters.	Core statistics view

The phrase “table cache hit ratio” or “index cache hit ratio” is shorthand. These views measure PostgreSQL shared-buffer hits versus reads. A read may still be satisfied by the operating-system page cache, so it is not a direct physical-disk hit ratio.

3. Prerequisites and Baseline Setup

3.1 Access requirements

- Use a monitoring role with least privilege. The built-in `pg_read_all_stats` role can see statistics that ordinary users cannot see for other sessions.
- Use superuser only when required for extension installation or privileged diagnostics.
- Install supplied extensions through the PostgreSQL packages for the same PostgreSQL major version.

3.2 Enable `pg_stat_statements`

`pg_stat_statements` needs shared memory and therefore must be included in `shared_preload_libraries`. PostgreSQL 18 computes query IDs automatically with `compute_query_id=auto` when required by `pg_stat_statements`.

```
-- postgresql.conf
shared_preload_libraries = 'pg_stat_statements'
compute_query_id = auto

-- Restart PostgreSQL after changing shared_preload_libraries.

-- Then, in every database where the SQL objects are needed:
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

3.3 Enable `pgstattuple` when physical bloat analysis is needed

```
CREATE EXTENSION IF NOT EXISTS pgstattuple;
```

```
-- Confirm installed extensions
SELECT extname, extversion
FROM pg_extension
WHERE extname IN ('pg_stat_statements','pgstattuple');
```

Security: Detailed pgstattuple functions have restricted execution privileges by default. Prefer controlled membership in pg_stat_scan_tables instead of broad superuser access.

3.4 Know the statistics age

```
SELECT datname, stats_reset
FROM pg_stat_database
WHERE datname = current_database();
```

Do not compare cumulative counters across systems or time windows without considering stats_reset, server uptime and workload duration.

4. Core Table and I/O Statistics

4.1 pg_stat_user_tables - table activity and maintenance

```
SELECT schemaname, relname,
       seq_scan, idx_scan,
       n_live_tup, n_dead_tup,
       n_tup_ins, n_tup_upd, n_tup_del, n_tup_hot_upd,
       last_autovacuum, autovacuum_count,
       last_autoanalyze, autoanalyze_count
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 20;
```

Use this view to find heavily modified tables, scan patterns and maintenance history. n_live_tup and n_dead_tup are estimates, so do not treat them as exact tuple counts.

4.2 Candidate tables with high estimated dead-tuple percentage

```
SELECT schemaname, relname, n_live_tup, n_dead_tup,
       round(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS dead_pct,
       last_autovacuum
FROM pg_stat_user_tables
WHERE n_dead_tup > 0
ORDER BY dead_pct DESC NULLS LAST, n_dead_tup DESC;
```

Interpretation: A high estimated dead percentage is a signal to investigate autovacuum behavior, update/delete rate, long-running transactions and table growth. It is not by itself proof that VACUUM FULL is required.

4.3 pg_statio_user_tables - shared-buffer effectiveness

```
SELECT schemaname, relname,
       heap_blks_read, heap_blks_hit,
       round(100.0 * heap_blks_hit / NULLIF(heap_blks_hit + heap_blks_read, 0), 2) AS heap_buffer_hit_pct
FROM pg_statio_user_tables
WHERE heap_blks_hit + heap_blks_read > 0
```

```
ORDER BY heap_blks_read DESC
LIMIT 20;
```

This can highlight relations driving PostgreSQL reads. The result should be interpreted with `pg_stat_io` and OS/cloud storage metrics; it is not a direct storage-device cache ratio.

4.4 pgstattuple() - physical tuple-space inspection

```
SELECT * FROM pgstattuple('public.orders'::regclass);
```

Important output includes `table_len`, `tuple_count`, `tuple_percent`, `dead_tuple_count`, `dead_tuple_percent`, `free_space` and `free_percent`. The function scans the relation and its result is accumulated page by page, so it is not an instantaneous snapshot under concurrent writes.

5. Index Statistics and Index Health

5.1 pg_stat_user_indexes - usage

```
SELECT schemaname, relname, indexrelname,
       idx_scan, last_idx_scan, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes
ORDER BY idx_scan ASC, indexrelname;
```

Do not drop blindly: `idx_scan = 0` does not automatically mean an index is useless. Consider statistics age, seasonal/reporting workloads, constraints, primary/unique indexes, standby usage, maintenance jobs and emergency queries before removal.

5.2 pg_statio_user_indexes - index buffer reads/hits

```
SELECT schemaname, relname, indexrelname,
       idx_blks_read, idx_blks_hit,
       round(100.0 * idx_blks_hit / NULLIF(idx_blks_hit + idx_blks_read, 0), 2) AS buffer_hit_pct
FROM pg_statio_user_indexes
WHERE idx_blks_hit + idx_blks_read > 0
ORDER BY idx_blks_read DESC;
```

5.3 pgstatindex() - B-tree physical structure

```
SELECT * FROM pgstatindex('public.orders_created_at_idx'::regclass);
```

Use it for B-tree indexes. Useful fields include `tree_level`, `index_size`, `leaf_pages`, `empty_pages`, `deleted_pages`, `avg_leaf_density` and `leaf_fragmentation`. High fragmentation is only one signal; consider index size, workload pattern and actual query benefit before `REINDEX`.

5.4 Index validity and metadata

```
SELECT n.nspname AS schema_name,
       t.relname AS table_name,
       i.relname AS index_name,
       x.indisunique, x.indisprimary, x.indisvalid, x.indisready,
       pg_get_indexdef(i.oid) AS index_definition
FROM pg_index x
JOIN pg_class i ON i.oid = x.indexrelid
JOIN pg_class t ON t.oid = x.indrelid
JOIN pg_namespace n ON n.oid = t.relnamespace
```

```
WHERE n.nspname NOT IN ('pg_catalog','information_schema')
ORDER BY 1,2,3;
```

5.5 Simple duplicate-definition candidate check

```
WITH idx AS (
  SELECT indrelid, indexrelid,
         indkey, indclass, indcollation, indoption, indexprs, indpred
  FROM pg_index
  WHERE indisvalid
)
SELECT a.indrelid::regclass AS table_name,
       a.indexrelid::regclass AS index_1,
       b.indexrelid::regclass AS index_2
FROM idx a
JOIN idx b ON a.indrelid = b.indrelid
             AND a.indexrelid < b.indexrelid
             AND a.indkey = b.indkey
             AND a.indclass = b.indclass
             AND a.indcollation = b.indcollation
             AND a.indoption = b.indoption
             AND a.indexprs IS NOT DISTINCT FROM b.indexprs
             AND a.indpred IS NOT DISTINCT FROM b.indpred
ORDER BY 1,2,3;
```

Caution: This query identifies strong duplicate candidates, not an automatic drop list. Review uniqueness, constraints, INCLUDE columns, access method, storage options and operational requirements before changing indexes.

6. Sessions, Locks and Blocking

6.1 pg_stat_activity - active and waiting sessions

```
SELECT pid, username, datname, application_name, client_addr,
       state, wait_event_type, wait_event,
       now() - query_start AS query_age,
       now() - xact_start AS xact_age,
       query
FROM pg_stat_activity
WHERE pid <> pg_backend_pid()
ORDER BY query_start NULLS LAST;
```

6.2 Long-running active queries

```
SELECT pid, username, application_name,
       now() - query_start AS runtime,
       wait_event_type, wait_event, query
FROM pg_stat_activity
WHERE state = 'active'
      AND query_start < now() - interval '5 minutes'
ORDER BY query_start;
```


6.3 Blocking tree using pg_blocking_pids()

```
SELECT blocked.pid AS blocked_pid,
       blocked.username AS blocked_user,
       pg_blocking_pids(blocked.pid) AS blocking_pids,
       now() - blocked.query_start AS blocked_for,
       blocked.query AS blocked_query
FROM pg_stat_activity blocked
WHERE cardinality(pg_blocking_pids(blocked.pid)) > 0
ORDER BY blocked.query_start;
```

6.4 Blocker details

```
SELECT b.pid AS blocked_pid,
       blocker.pid AS blocker_pid,
       blocker.username AS blocker_user,
       blocker.state AS blocker_state,
       now() - blocker.xact_start AS blocker_xact_age,
       blocker.query AS blocker_query
FROM pg_stat_activity b
CROSS JOIN LATERAL unnest(pg_blocking_pids(b.pid)) AS p(blocker_pid)
JOIN pg_stat_activity blocker ON blocker.pid = p.blocker_pid;
```

Operational rule: Do not terminate sessions only because they appear old. Confirm business impact, transaction state, lock relationship and application retry behavior. `pg_cancel_backend()` requests query cancellation; `pg_terminate_backend()` ends the session and is more disruptive.

7. Query Performance with pg_stat_statements

7.1 Top SQL by total execution time

```
SELECT queryid, calls,
       round(total_exec_time::numeric, 2) AS total_exec_ms,
       round(mean_exec_time::numeric, 2) AS mean_exec_ms,
       rows,
       shared_blks_hit, shared_blks_read,
       temp_blks_read, temp_blks_written,
       left(query, 300) AS query_sample
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 20;
```

7.2 Top SQL by average execution time

```
SELECT queryid, calls,
       round(mean_exec_time::numeric, 2) AS mean_exec_ms,
       round(total_exec_time::numeric, 2) AS total_exec_ms,
       left(query, 300) AS query_sample
FROM pg_stat_statements
WHERE calls >= 10
ORDER BY mean_exec_time DESC
LIMIT 20;
```

7.3 SQL causing the most temporary I/O

```
SELECT queryid, calls, temp_blks_read, temp_blks_written,
       round(temp_blk_read_time::numeric,2) AS temp_read_ms,
       round(temp_blk_write_time::numeric,2) AS temp_write_ms,
       left(query,300) AS query_sample
FROM pg_stat_statements
WHERE temp_blks_read + temp_blks_written > 0
ORDER BY temp_blks_written DESC
LIMIT 20;
```

Timing caveat: Block read/write timing columns depend on track_io_timing. If I/O timing is disabled, those timing fields do not provide useful elapsed-I/O measurements.

7.4 Reset only with an operational reason

```
-- Requires appropriate privilege. Reset destroys the previous cumulative baseline.
SELECT pg_stat_statements_reset();
```

8. Database, Vacuum and Index Build Progress

8.1 pg_stat_database - database-level health

```
SELECT datname, numbackends, xact_commit, xact_rollback,
       blks_read, blks_hit, temp_files, pg_size_pretty(temp_bytes) AS temp_bytes,
       deadlocks, checksum_failures, sessions, sessions_abandoned,
       sessions_fatal, stats_reset
FROM pg_stat_database
WHERE datname IS NOT NULL
ORDER BY datname;
```

8.2 pg_stat_progress_vacuum

```
SELECT pid, datname, relid::regclass AS relation, phase,
       heap_blks_total, heap_blks_scanned, heap_blks_vacuumed,
       index_vacuum_count, num_dead_item_ids, max_dead_item_ids
FROM pg_stat_progress_vacuum;
```

8.3 pg_stat_progress_create_index

```
SELECT pid, datname, relid::regclass AS table_name,
       index_relid::regclass AS index_name, command, phase,
       lockers_total, lockers_done,
       blocks_total, blocks_done, tuples_total, tuples_done
FROM pg_stat_progress_create_index;
```

Progress counters vary by phase. A zero or unchanged tuple counter does not necessarily mean the operation is hung; first check the current phase and wait event.

9. Replication and WAL Monitoring

9.1 pg_stat_replication - primary-side streaming status

```
SELECT pid, application_name, client_addr, state, sync_state,
       sent_lsn, write_lsn, flush_lsn, replay_lsn,
```

```
pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS replay_gap,
write_lag, flush_lag, replay_lag
FROM pg_stat_replication
ORDER BY application_name;
```

Lag interpretation: write_lag, flush_lag and replay_lag are measurements of recent WAL processing behavior and are not guaranteed to be a continuously meaningful “standby is N seconds behind” metric. LSN byte gaps and application-level recovery objectives should also be monitored.

9.2 Standby-side WAL receiver

```
SELECT status, sender_host, sender_port, slot_name,
       written_lsn, flushed_lsn, received_tli, latest_end_lsn, latest_end_time
FROM pg_stat_wal_receiver;
```

9.3 pg_stat_wal - WAL generation

```
SELECT wal_records, wal_fpi, pg_size_pretty(wal_bytes::bigint) AS wal_bytes,
       wal_buffers_full, wal_write, wal_sync,
       wal_write_time, wal_sync_time, stats_reset
FROM pg_stat_wal;
```

Use WAL growth with checkpoint, write-heavy workload and replication metrics. wal_bytes is instance-wide since the WAL statistics reset; it is not broken down by database.

10. Additional High-Value PostgreSQL 18 Views

View	Why it matters
pg_stat_io	Cluster-wide I/O activity by backend type, I/O object and context. Strong companion to pg_statio_* views.
pg_stat_checkpointer	Checkpoint counts, timing and buffer-writing statistics for the checkpointer.
pg_stat_bgwriter	Background writer activity.
pg_stat_archiver	WAL archiver successes/failures and last archived/failed WAL file.
pg_stat_replication_slots	Replication-slot statistics and spill/stream behavior where applicable.
pg_stat_wal_receiver	Standby-side receiver status and LSN positions.
pg_stat_database_conflicts	Recovery conflicts on standby databases.
pg_stat_ssl	TLS status and protocol/cipher information per connection.
pg_stat_progress_analyze	Live ANALYZE progress.
pg_stat_progress_cluster	Live CLUSTER and VACUUM FULL progress.
pg_stat_progress_copy	Live COPY progress.
pg_stat_progress_basebackup	Base-backup progress for WAL sender processes.
pg_stat_slru	SLRU statistics such as blocks zeroed/hit/read/written.

View	Why it matters
pg_stat_user_functions	Execution counts and timing for tracked user functions when function tracking is enabled.

10.1 pg_stat_io example

```
SELECT backend_type, object, context,
       reads, read_bytes, read_time,
       writes, write_bytes, write_time,
       writebacks, writeback_time, extends, hits, evictions
FROM pg_stat_io
ORDER BY read_bytes DESC NULLS LAST;
```

PostgreSQL 18 note: PostgreSQL 18 exposes byte counters in pg_stat_io in addition to operation counts. Use the exact columns available in your installed minor release and verify with \d+ pg_stat_io.

10.2 Archiver status

```
SELECT archived_count, last_archived_wal, last_archived_time,
       failed_count, last_failed_wal, last_failed_time, stats_reset
FROM pg_stat_archiver;
```

11. Practical Performance Tuning Workflow

Step	Action	What to do
1	Confirm incident scope	Is the problem query-specific, database-wide or host-wide? Check application symptoms and OS/cloud CPU, memory, I/O latency and network.
2	Inspect live activity	Use pg_stat_activity, wait_event_type and wait_event. Identify active work, idle-in-transaction sessions and long-running transactions.
3	Check blocking	Use pg_blocking_pids() and pg_locks. Resolve blockers only after understanding the transaction and application impact.
4	Rank SQL	Use pg_stat_statements by total_exec_time, mean_exec_time, shared reads, temp I/O and calls.
5	Inspect plans	Use EXPLAIN first; use EXPLAIN (ANALYZE, BUFFERS) only when safe because it executes the statement.
6	Inspect table/index behavior	Use pg_stat_user_tables, pg_stat_user_indexes and pg_statio_* to identify scan patterns and hot relations.
7	Check maintenance	Review dead-tuple estimates, autovacuum/analyze history and progress views.
8	Check WAL/replication	Use pg_stat_wal, pg_stat_replication, pg_stat_wal_receiver and slot/archiver views where applicable.
9	Correlate with	Confirm storage latency, IOPS, throughput, memory pressure, CPU

Step	Action	What to do
	OS/storage	saturation and network behavior.
10	Change one thing and validate	Record baseline, implement a controlled change, compare before/after metrics and keep a rollback path.

11.1 Safe EXPLAIN sequence

```
-- 1. Planner-only; does not execute the SELECT.
EXPLAIN (VERBOSE, COSTS, SETTINGS)
SELECT ...;

-- 2. Executes the statement. Use only when safe.
EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS, SUMMARY)
SELECT ...;
```

12. Production Troubleshooting Scenarios

12.1 Scenario - application latency suddenly increases

- Symptom: API/database response time rises while connections remain available.
- Check: `pg_stat_activity` for active query age, wait events and idle-in-transaction sessions.
- Check: blocking relationships with `pg_blocking_pids()`.
- Check: `pg_stat_statements` for statements with rising total/mean execution time and block reads.
- Check: OS/cloud CPU and storage latency at the same timestamp.
- Action: address the confirmed bottleneck; do not assume every latency event is caused by SQL plans.
- Validate: compare response time, active wait events and query statistics after the change.

12.2 Scenario - table growth after heavy UPDATE/DELETE workload

- Check `n_dead_tup`, `last_autovacuum` and `autovacuum_count` in `pg_stat_user_tables`.
- Check for long-running/idle-in-transaction sessions that can delay tuple cleanup.
- Use `pgstattuple()` during a controlled window if physical tuple/free-space detail is needed.
- Review autovacuum thresholds and workload; do not jump directly to `VACUUM FULL` because it requires a stronger lock and rewrites the table.
- Validate table size, dead tuple trend, vacuum behavior and query latency afterward.

12.3 Scenario - index appears unused

- Confirm `stats_reset` and workload observation window.
- Check `idx_scan/last_idx_scan` in `pg_stat_user_indexes`.
- Confirm whether the index backs a PRIMARY KEY or UNIQUE constraint.
- Check application/reporting/seasonal jobs and standby workload.
- Compare the definition with other indexes and query plans before considering removal.

12.4 Scenario - standby is falling behind

- Primary: inspect `pg_stat_replication` state, LSN positions and byte gap.
- Standby: inspect `pg_stat_wal_receiver` and replay activity.
- Check WAL generation rate in `pg_stat_wal`.
- Check network throughput/latency and standby storage write/replay performance.

- Check for replay conflicts and long-running standby queries when hot standby is used.
- Validate that the measured lag meets the defined RPO/RTO or service objective.

13. Production Readiness Checklist

- ☐ `pg_stat_statements` is enabled only where required and its memory/retention settings are sized for the workload.
- ☐ Monitoring role uses least privilege, typically `pg_read_all_stats` where appropriate.
- ☐ Statistics reset times and PostgreSQL restart times are tracked so cumulative metrics have context.
- ☐ Long-running transactions and idle-in-transaction sessions are alerted on.
- ☐ Blocking chains and deadlocks are monitored.
- ☐ Top SQL is monitored by total time, mean time, calls, buffer reads and temporary I/O.
- ☐ Autovacuum and autoanalyze activity is monitored on high-churn tables.
- ☐ Index usage is reviewed with a long enough observation window before removal decisions.
- ☐ Checkpoint, WAL generation, archiving and replication are monitored where relevant.
- ☐ PostgreSQL metrics are correlated with CPU, memory, storage latency/IOPS/throughput and network.
- ☐ Any performance change has a baseline, rollback plan and post-change validation.
- ☐ High-impact commands such as `VACUUM FULL`, `REINDEX` on critical indexes, session termination and statistics resets have change-control procedures.

14. Operational Cheat Sheet

Task	Quick query
Current active sessions	<code>SELECT * FROM pg_stat_activity WHERE state='active';</code>
Blocked sessions	<code>SELECT pid, pg_blocking_pids(pid), query FROM pg_stat_activity WHERE cardinality(pg_blocking_pids(pid))>0;</code>
Table dead-tuple estimates	<code>SELECT relname,n_live_tup,n_dead_tup,last_autovacuum FROM pg_stat_user_tables ORDER BY n_dead_tup DESC;</code>
Index usage	<code>SELECT relname,indexrelname,idx_scan,last_idx_scan FROM pg_stat_user_indexes ORDER BY idx_scan;</code>
Vacuum progress	<code>SELECT * FROM pg_stat_progress_vacuum;</code>
Index build progress	<code>SELECT * FROM pg_stat_progress_create_index;</code>
Replication senders	<code>SELECT * FROM pg_stat_replication;</code>
WAL statistics	<code>SELECT * FROM pg_stat_wal;</code>
I/O statistics	<code>SELECT * FROM pg_stat_io;</code>
Top SQL	<code>SELECT queryid,calls,total_exec_time,mean_exec_time,query FROM pg_stat_statements ORDER BY total_exec_time DESC LIMIT 20;</code>

15. Official References

Reference	Official URL
PostgreSQL 18 - Monitoring Database Activity	https://www.postgresql.org/docs/18/monitoring.html
PostgreSQL 18 - Cumulative Statistics System	https://www.postgresql.org/docs/18/monitoring-stats.html
PostgreSQL 18 - Run-time Statistics Configuration	https://www.postgresql.org/docs/18/runtime-config-statistics.html
PostgreSQL 18 - pg_stat_statements	https://www.postgresql.org/docs/18/pgstatstatements.html
PostgreSQL 18 - pgstattuple / pgstatindex	https://www.postgresql.org/docs/18/pgstattuple.html
PostgreSQL 18 - pg_index System Catalog	https://www.postgresql.org/docs/18/catalog-pg-index.html
PostgreSQL 18 - pg_indexes System View	https://www.postgresql.org/docs/18/view-pg-indexes.html

15.1 Final takeaway

The 15-item matrix is a strong starting point, but production performance analysis requires context. The most important points are to distinguish extension functions from core views, interpret cache-hit ratios as PostgreSQL buffer statistics rather than physical-disk ratios, treat index “fragmentation” carefully, and combine replication lag fields with LSN gaps and infrastructure metrics. PostgreSQL 18 pg_stat_io is also an important modern addition for system-wide I/O analysis.